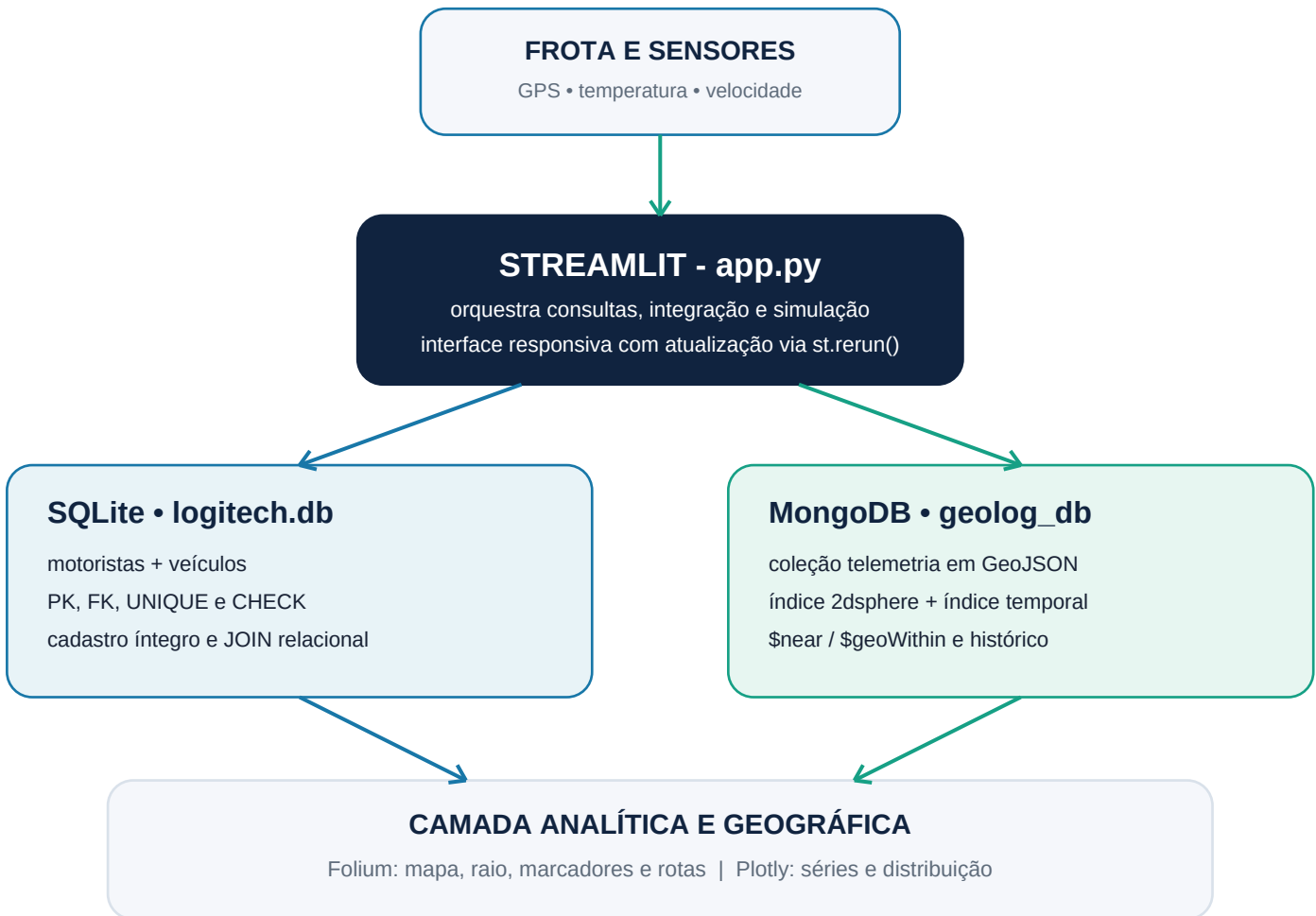


Persistência poliglota para telemetria

Um dashboard, dois modelos de dados, cada um aplicado ao seu ponto forte.

Diagrama de arquitetura



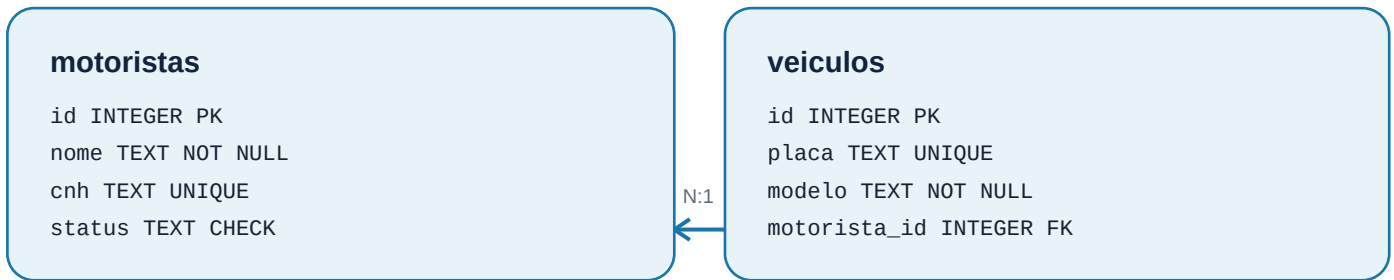
Responsabilidades

- SQLite preserva integridade cadastral: cada veículo referencia um motorista existente e placas/CNHs são únicas.
- MongoDB absorve escrita frequente e documentos flexíveis, mantendo coordenadas no padrão GeoJSON [longitude, latitude].
- O `veiculo_id` é a chave de integração lógica: a aplicação consulta a última telemetria e enriquece o resultado com motorista, placa e modelo.

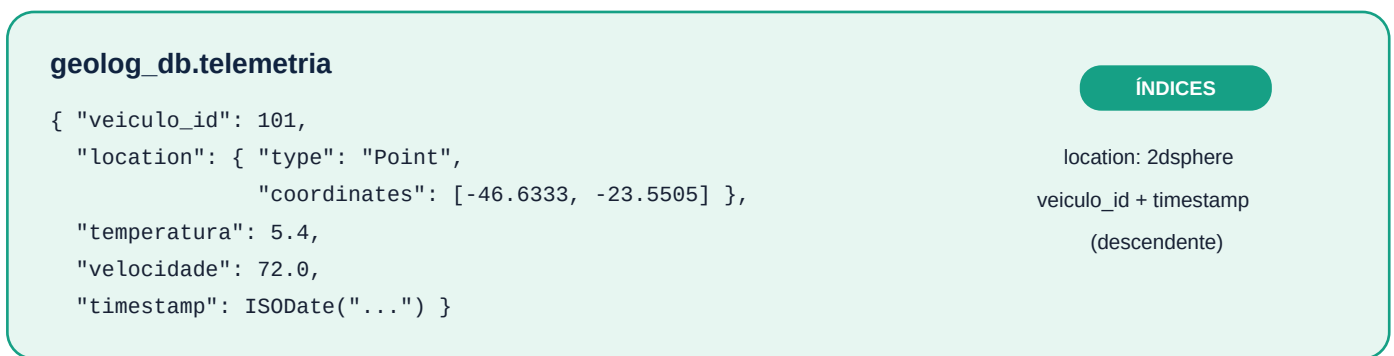
Decisão central

Dados estáveis e relacionais ficam no SQLite; eventos volumosos, temporais e espaciais ficam no MongoDB.

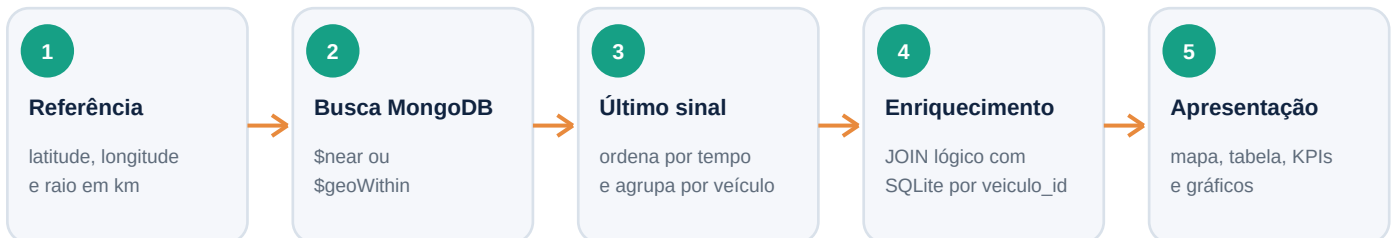
Modelo relacional - SQLite



Documento NoSQL - MongoDB



Fluxo da consulta geoespacial



Regra de proximidade

A consulta espacial identifica documentos candidatos no índice 2dsphere. Em seguida, o app seleciona o documento mais recente de cada veículo e valida a distância Haversine. Essa segunda validação impede que um ponto histórico antigo faça um veículo atualmente distante aparecer no resultado.

Consistência da integração

O MongoDB guarda o identificador externo veiculo_id; o cadastro mestre e suas regras permanecem no SQLite.

Componentes do dashboard

KPIs	Mapa Folium	Plotly
Frota ativa, temperatura média, alertas de velocidade e veículos no raio.	Referência, círculo de busca, cluster, alertas visuais e rotas recentes.	Histórico térmico por placa e distribuição do status dos motoristas.

Simulador em tempo real

Botão “Simular Movimentação”

1. Lê a posição mais recente → 2. Aplica pequena variação aleatória em GPS e sensores
3. Grava um novo documento por veículo → 4. Executa `st.rerun()` e recompõe todo o painel

Como executar

```
pip install streamlit streamlit-folium folium plotly pandas pymongo
docker run -d --name geolog-mongo -p 27017:27017 mongo:7
cd Desafio
streamlit run app.py
```

Validação e comportamento esperado

- Na primeira execução, o app cria as duas tabelas SQLite, aplica chaves/restrições e inclui oito cadastros de demonstração.
- Ao conectar no MongoDB, cria os índices, inclui histórico inicial se a coleção estiver vazia e mantém os registros existentes nas execuções seguintes.
- Sem MongoDB, o dashboard não quebra: apresenta diagnóstico e continua exibindo o cadastro relacional disponível.
- O código usa consultas parametrizadas no SQLite, timestamps UTC e limites nas consultas de mapa/histórico.

Entregáveis

app.py (aplicação única) • logitech.db (gerado automaticamente) • relatório técnico em PDF